

Software Engineering

- 1) Introduction
- 2) Software Development Life cycle **
- 3) Requirement analysis (SRS)
- 4) Software Project Management * (COCOMO)
- 5) Software Design (Coupling, Cohesion, UML, DFD, class diagram)
- 6) Coding and Testing **
- 7) Maintenance
- 8) Quality Mgmt, Reuse

Important buzzwords of software engineering

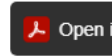
Fault, error, failure, bug

Fault



- Fault is a defect within the system
- Examples:
 - Software bug
 - Random hardware fault
 - Memory bit “stuck”
 - Omission or commission fault in data transfer
 - Etc.

Error



- Error is a deviation from the required operation of system or subsystem
- A fault may lead to an error, i.e., error is a mechanism by which the fault becomes apparent
- Fault may stay dormant for a long time before it manifests itself as an error:
- Example:
 - memory bit got stuck but CPU does not access this data
 - Software “bug” in a subroutine is not “visible” while the subroutine is not called

Failure



- A system failure occurs when the system fails to perform its required function
- Presence of an error might cause a whole system to deviate from its required operation
- One of the goals of safety-critical systems is that error should not result in system failure

HOW ARE THEY IDENTIFIED

Testing is the process of identifying **defects**, where a defect is any variance between actual and expected results.

“A mistake in coding is called **Error**, error found by tester is called Defect, defect accepted by development team then it is **called Bug**, build does not meet the requirements then it is Failure.”

Defect can be categorized into the following:

Wrong: When requirements are implemented not in the right way. This defect is a variance from the given specification. It is Wrong!

Missing: A requirement of the customer that was not fulfilled. This is a variance from the specifications, an indication that a specification was not implemented, or a requirement of the customer was not noted correctly.

Extra: A requirement incorporated into the product that was not given by the end customer. This is always a variance from the specification, but may be an attribute desired by the user of the product. However, it is considered a defect because it's a variance from the existing requirements.

(1) Software & Software Engineering

Software:

- ☐ Computer software is the product that software professionals build and then support over the long term.
- ☐ It encompasses programs that execute within a computer of any size and architecture, descriptive information in both hard copy and virtual forms.

Characteristics of software:

[1] Software is developed or engineered; it is not manufactured in the classical sense:

- ☐ In both activities, high quality is achieved through good design.

[2] Software doesn't "wear out."

- ☐ Hardware components suffer from the growing effects of dust, vibration, abuse, temperature extremes, and many other environmental maladies. Stated simply, the hardware begins to wear out.
- ☐ When a hardware component wears out, it is replaced by a spare part.
- ☐ There are no software spare parts.
- ☐ Every software failure indicates an error in design or in the process through which design was translated into machine executable code.

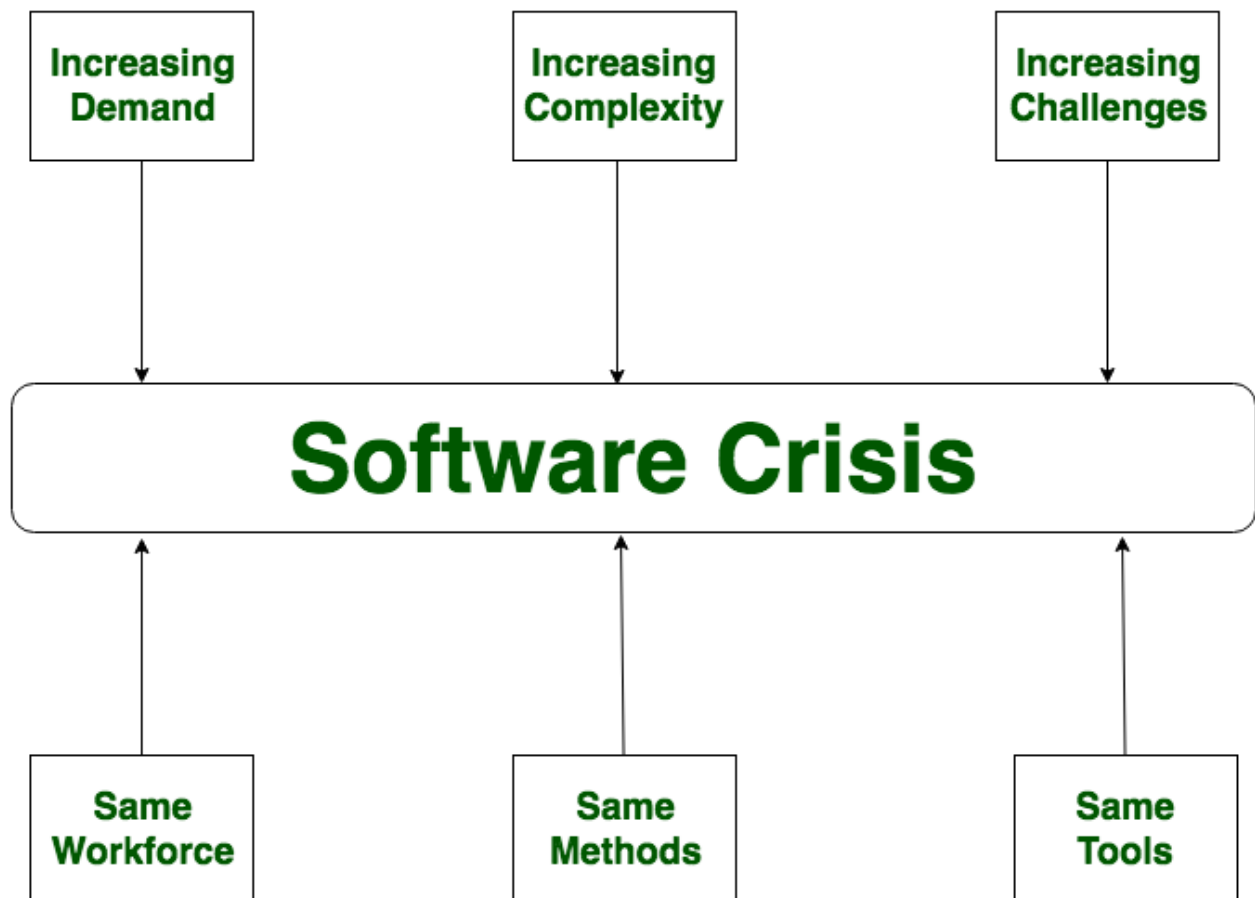
[3] Although the industry is moving toward component-based construction, most software continues to be custom built.

- ☐ A software component should be designed and implemented so that it can be reused in many different programs.

SOFTWARE CRISIS

The difficulty of writing the code for a computer program which is correct and understandable is referred to as software crisis.

or Software crisis is also referred to as inability to hire enough qualified programmers.



Causes of Software Crisis:

- The cost of owning and maintaining software was as expensive as developing the software
- At that time Projects was running over-time
- At that time Software was very inefficient
- The quality of software was low quality
- Software often did not meet requirements

- The average software project overshoots its schedule by half
- At that time Software was never delivered

Solution of Software Crisis:

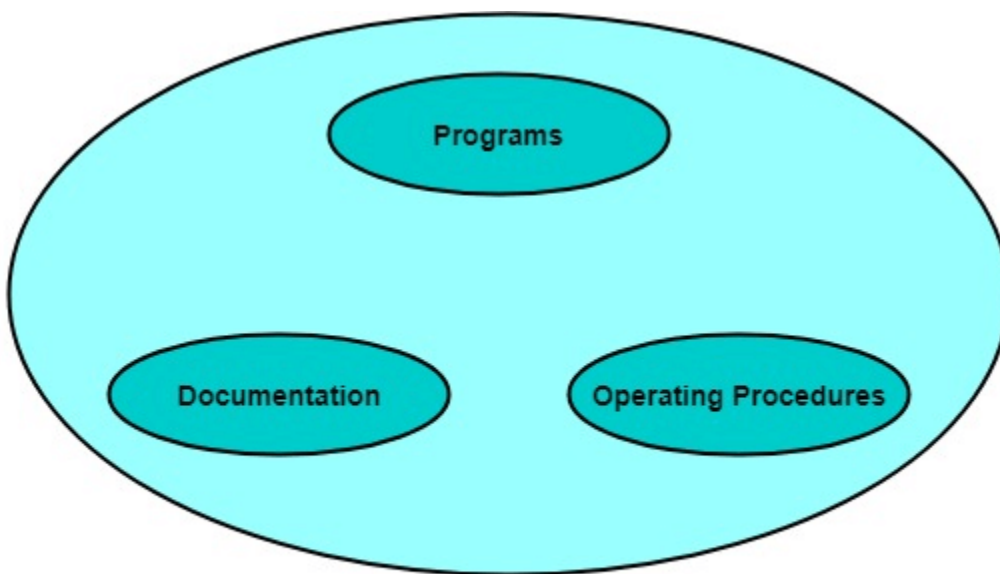
There is no single solution to the crisis. one possible solution of software crisis is *Software Engineering* because software engineering is a systematic, disciplined and quantifiable approach. For preventing software crisis, there are some guidelines:

- Reduction in software over-budget
- The quality of software must be high
- Less time needed for software project
- Experience working team member on software project
- Software must be delivered

Program vs. Software

Software is more than programs. Any program is a subset of software, and it becomes software only if documentation & operating procedures manuals are prepared.

There are three components of the software as shown in fig:



Software= Program + Documentation + Operating Procedures

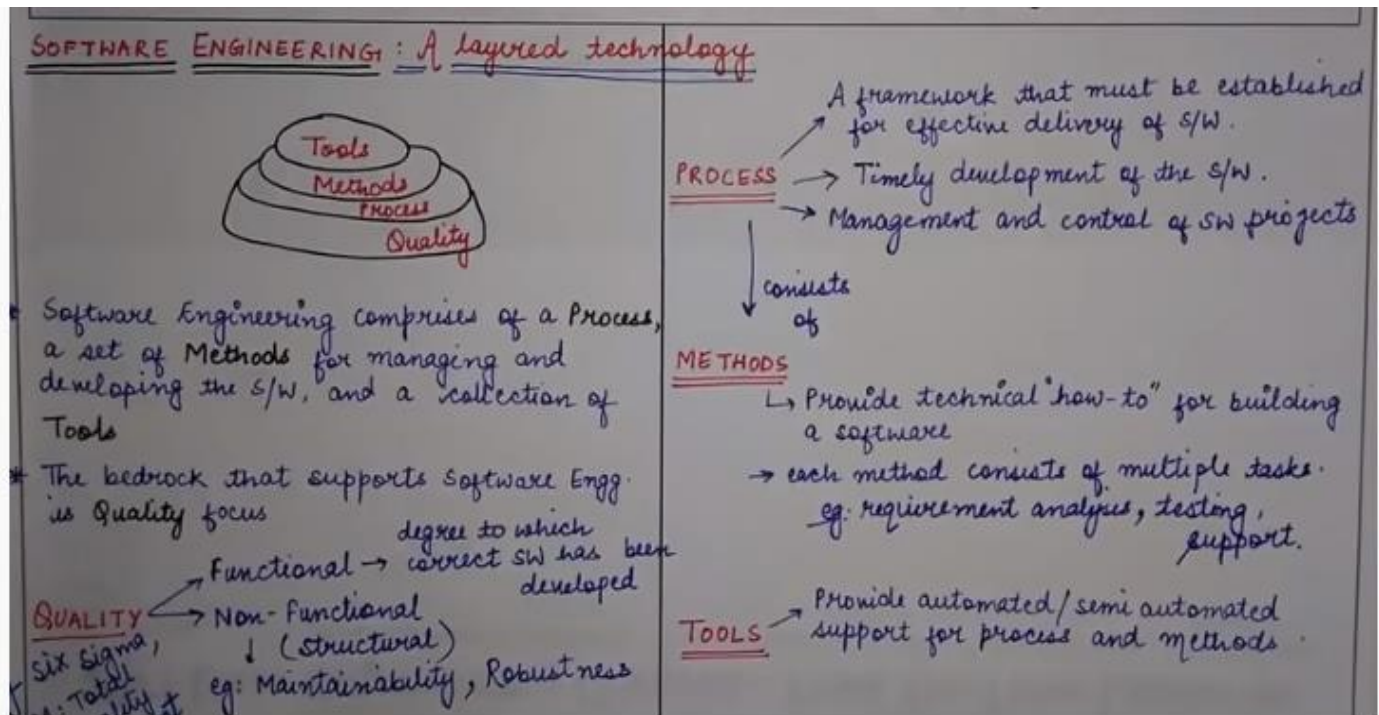
Fig:Components of Software

1. Program: Program is a combination of source code & object code.

2. Documentation: Documentation consists of different types of manuals. Examples of documentation manuals are: Data Flow Diagram, Flow Charts, ER diagrams, etc.

Software Crisis

1. **Size:** Software is becoming more expensive and more complex with the growing complexity and expectation out of software. For example, the code in the consumer product is doubling every couple of years.
2. **Quality:** Many software products have poor quality, i.e., the software products defects after putting into use due to ineffective testing technique. For example, Software testing typically finds 25 errors per 1000 lines of code.
3. **Cost:** Software development is costly i.e. in terms of time taken to develop and the money involved. For example, Development of the FAA's Advanced Automation System cost over \$700 per lines of code.
4. **Delayed Delivery:** Serious schedule overruns are common. Very often the software takes longer than the estimated time to develop, which in turn leads to cost shooting up. For example, one in four large-scale development projects is never completed.



Software Processes

A software process is the set of activities and associated outcome that produce a software product. Software engineers mostly carry out these activities. These are four key process activities, which are common to all software processes. These activities are:

1. **Software specifications:** The functionality of the software and constraints on its operation must be defined.
2. **Software development:** The software to meet the requirement must be produced.
3. **Software validation:** The software must be validated to ensure that it does what the customer wants.
4. **Software evolution:** The software must evolve to meet changing client needs.

The Software Process Model

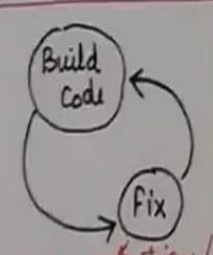
A software process model is a specified definition of a software process, which is presented from a particular perspective. Models, by their nature, are a simplification, so a

software process model is an abstraction of the actual process, which is being described. Process models may contain activities, which are part of the software process, software product, and the roles of people involved in software engineering.

There are several various general models or paradigms of software development:

1. **The waterfall approach:** This takes the above activities and produces them as separate process phases such as requirements specification, software design, implementation, testing, and so on. After each stage is defined, it is "signed off" and development goes onto the following stage.
2. **Evolutionary development:** This method interleaves the activities of specification, development, and validation. An initial system is rapidly developed from a very abstract specification.
3. **Formal transformation:** This method is based on producing a formal mathematical system specification and transforming this specification, using mathematical methods to a program. These transformations are 'correctness preserving.' This means that you can be sure that the developed programs meet its specification.

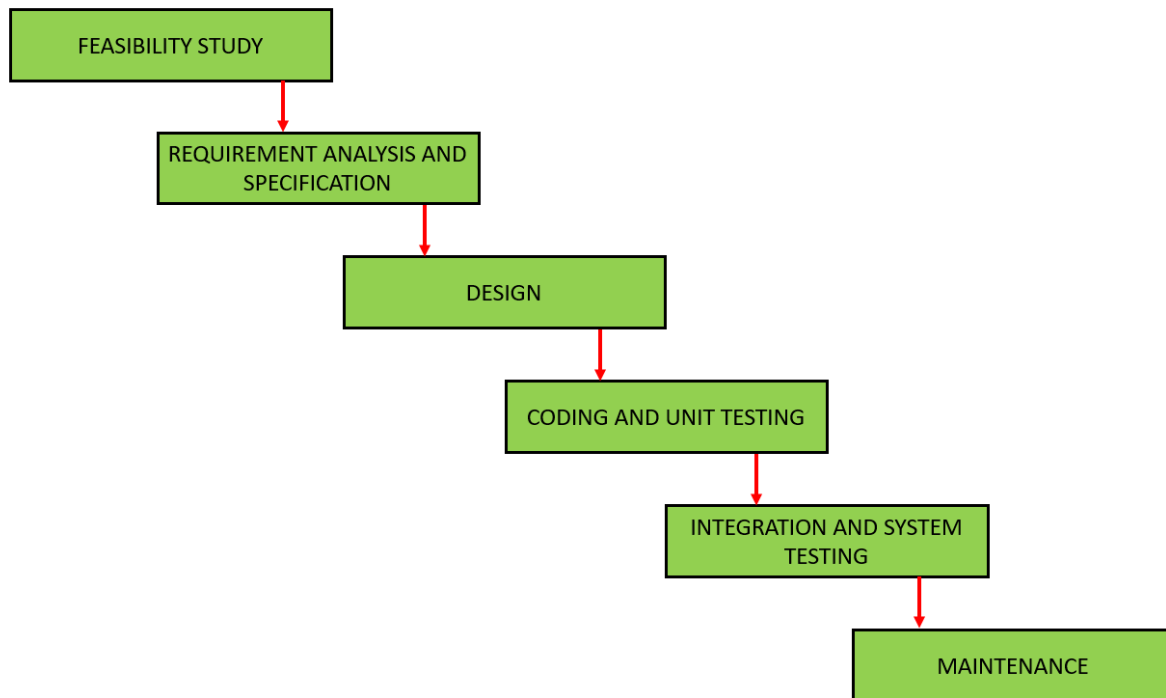
4. **System assembly from reusable components:** This method assumes the parts of the system already exist. The system development process target on integrating these parts rather than developing them from scratch.

SOFTWARE LIFE CYCLE MODELS	
The Process followed in the development of the SW depends upon the LIFE CYCLE MODEL chosen for development. Life Cycle Model defines the major phases during and after the S/W development process. <u>Types of Life Cycle Models</u> 1. BUILD AND FIX MODEL 2. WATERFALL MODEL 3. INCREMENTAL PROCESS MODELS A) ITERATIVE MODEL B) RAPID APPLICATION DEVELOPMENT <i>→ RAD</i> 4. EVOLUTIONARY MODELS A) PROTOTYPING MODEL B) SPIRAL MODEL 5. RATIONAL UNIFIED PROCESS MODEL	<u>1. Build And Fix Model</u>  • 2 Phase Model consisting of Writing Code → Fixing It • Product constructed <u>without any formal requirement specifications OR design</u>. <i>excess correction/ addition of new functionality</i> * Directly developers start building the product which is fixed (changed/reworked) as many times as the customer desires. <u>Disadvantages</u> 1. Not suitable for large size S/W 2. Cost is very high as compared to a structured approach 3. Code becomes unfixable very soon. 4. Maintenance is very difficult ↳ Lack of requirement & design documents.

Waterfall Model

Classical waterfall model divides the life cycle into a set of phases. This model considers that one phase can be started after completion of the previous phase. That is the output of one phase will be the input to the next phase. Thus the development process can be considered as a sequential flow in the waterfall. Here the phases do not overlap with each other. The different sequential

phases of the classical waterfall model are shown in the below figure:



Let us now learn about each of these phases in brief details:

1. **Feasibility Study:** The main goal of this phase is to determine whether it would be financially and technically feasible to develop the software. The feasibility study involves understanding the problem and then determine the various possible strategies to solve the problem. These different identified solutions are analyzed based on their benefits and drawbacks, The best solution is chosen and all the other phases are carried out as per this solution strategy.
2. **Requirements analysis and specification:** The aim of the requirement analysis and specification phase is to understand the exact requirements of the customer and document them properly. This phase consists of two different activities.
 - **Requirement gathering and analysis:** Firstly all the requirements regarding the software are gathered from the customer and then the gathered requirements are analyzed. The goal of the analysis part is to remove incompleteness (an incomplete requirement is one in which some parts of the actual requirements have been omitted) and inconsistencies (inconsistent requirement is one in which some part of the requirement contradicts with some other part).
 - **Requirement specification:** These analyzed requirements are documented in a software requirement specification (SRS) document. SRS document serves as a contract between development team and

customers. Any future dispute between the customers and the developers can be settled by examining the SRS document.

3. **Design:** The aim of the design phase is to transform the requirements specified in the SRS document into a structure that is suitable for implementation in some programming language.
4. **Coding and Unit testing:** In coding phase software design is translated into source code using any suitable programming language. Thus each designed module is coded. The aim of the unit testing phase is to check whether each module is working properly or not.
5. **Integration and System testing:** Integration of different modules are undertaken soon after they have been coded and unit tested. Integration of various modules is carried out incrementally over a number of steps. During each integration step, previously planned modules are added to the partially integrated system and the resultant system is tested. Finally, after all the modules have been successfully integrated and tested, the full working system is obtained and system testing is carried out on this. System testing consists three different kinds of testing activities as described below :

- **Alpha testing:** Alpha testing is the system testing performed by the development team.
 - **Beta testing:** Beta testing is the system testing performed by a friendly set of customers.
 - **Acceptance testing:** After the software has been delivered, the customer performed the acceptance testing to determine whether to accept the delivered software or to reject it.
6. **Maintainence:** Maintenance is the most important phase of a software life cycle. The effort spent on maintenance is the 60% of the total effort spent to develop a full software. There are basically three types of maintenance :
 - **Corrective Maintenance:** This type of maintenance is carried out to correct errors that were not discovered during the product development phase.
 - **Perfective Maintenance:** This type of maintenance is carried out to enhance the functionalities of the system based on the customer's request.
 - **Adaptive Maintenance:** Adaptive maintenance is usually required for porting the software to work in a new environment such as work on a new computer platform or with a new operating system.

Advantages of Classical Waterfall Model

Classical waterfall model is an idealistic model for software development. It is very simple, so it can be considered as the basis for other software development life cycle models. Below are some of the major advantages of this SDLC model:

- This model is very simple and is easy to understand.
- Phases in this model are processed one at a time.
- Each stage in the model is clearly defined.
- This model has very clear and well understood milestones.
- Process, actions and results are very well documented.
- Reinforces good habits: define-before- design, design-before-code.
- This model works well for smaller projects and projects where requirements are well understood.

Drawbacks of Classical Waterfall Model

Classical waterfall model suffers from various shortcomings, basically we can't use it in real projects, but we use other software development lifecycle models which are based on the classical waterfall model. Below are some major drawbacks of this model:

- **No feedback path:** In classical waterfall model evolution of a software from one phase to another phase is like a waterfall. It assumes that no error is ever committed by developers during any phases. Therefore, it does not incorporate any mechanism for error correction.
- **Difficult to accommodate change requests:** This model assumes that all the customer requirements can be completely and correctly defined at the beginning of the project, but actually customers' requirements keep on changing with time. It is difficult to accommodate any change requests after the requirements specification phase is complete.
- **No overlapping of phases:** This model recommends that new phase can start only after the completion of the previous phase. But in real projects, this can't be maintained. To increase the efficiency and reduce the cost, phases may overlap.

PROBLEMS WITH WATERFALL MODEL	WHEN TO USE WATERFALL MODEL
1. Expects Complete and Accurate Requirements early in s/w development process. <i>Not Realistic</i> 2. Working s/w is not available till very late in the s/w development cycle. → delay in error discovery. 3. Risk ↳ No assessment of risk 4. Change → NOT suitable for accommodating changing during development 5. Sequential Nature ↳ Not realistic in today's world AND this model is not suitable for large projects	Follows the idea of <u>Define</u> Before <u>Design</u> AND <u>Design</u> Before <u>Code</u> 1. When the requirements are very clearly understood & they will not change during SDLC. ↓ <i>stable</i> 2. Can be used by an organisation : - a) that has an experience in developing a particular kind of s/w. b) when it wants to build a new software based on an existing s/w.

Incremental Model

Incremental Model is a process of software development where requirements divided into multiple standalone modules of the software development cycle. In this model, each module goes through the requirements, design, implementation and testing phases. Every subsequent release of the module adds function to the previous release. The process continues until the complete system achieved.

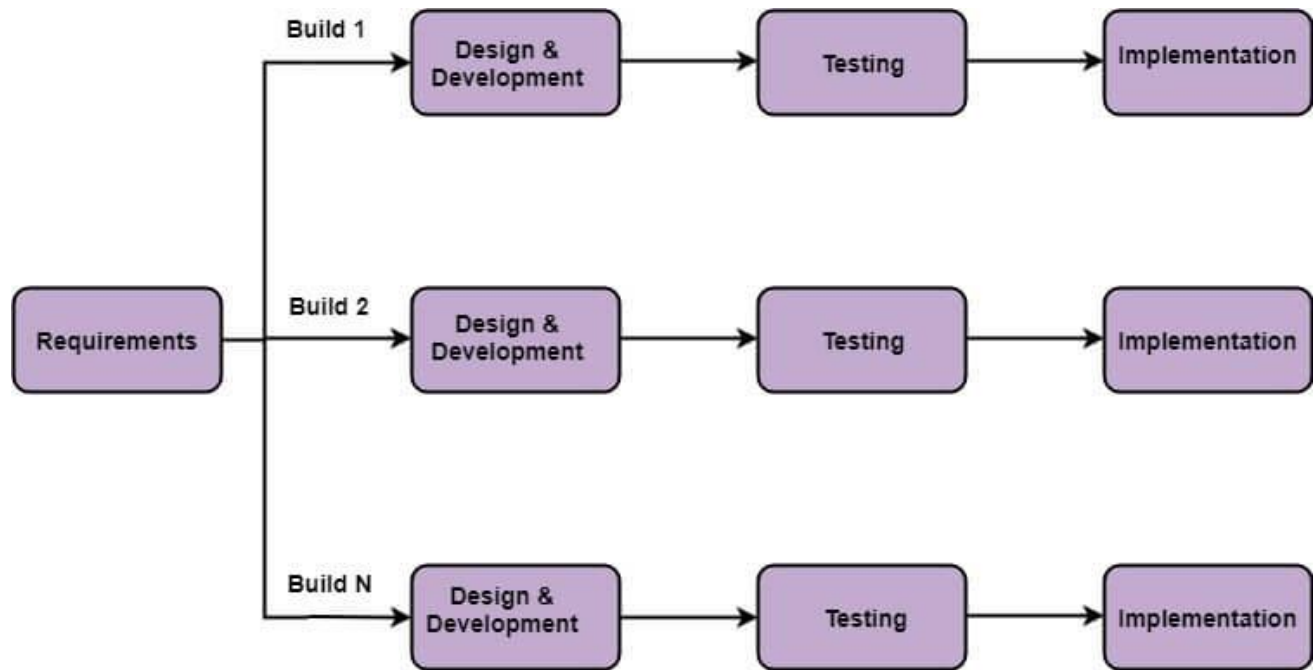


Fig: Incremental Model

The various phases of incremental model are as follows:

1. Requirement analysis: In the first phase of the incremental model, the product analysis expertise identifies the requirements. And the system functional requirements are understood by the requirement analysis team. To develop the software under the incremental model, this phase performs a crucial role.

2. Design & Development: In this phase of the Incremental model of SDLC, the design of the system functionality and the development method are finished with success. When software develops new practicality, the incremental model uses style and development phase.

3. Testing: In the incremental model, the testing phase checks the performance of each existing function as well as additional functionality. In the testing phase, the various methods are used to test the behavior of each task.

4. Implementation: Implementation phase enables the coding phase of the development system. It involves the final coding that design in the designing and development phase and tests the functionality in the testing phase. After completion of this phase, the number of the product working is enhanced and upgraded up to the final system product

When we use the Incremental Model?

- When the requirements are superior.
- A project has a lengthy development schedule.
- When Software team are not very well skilled or trained.
- When the customer demands a quick release of the product.
- You can develop prioritized requirements first.

Advantage of Incremental Model

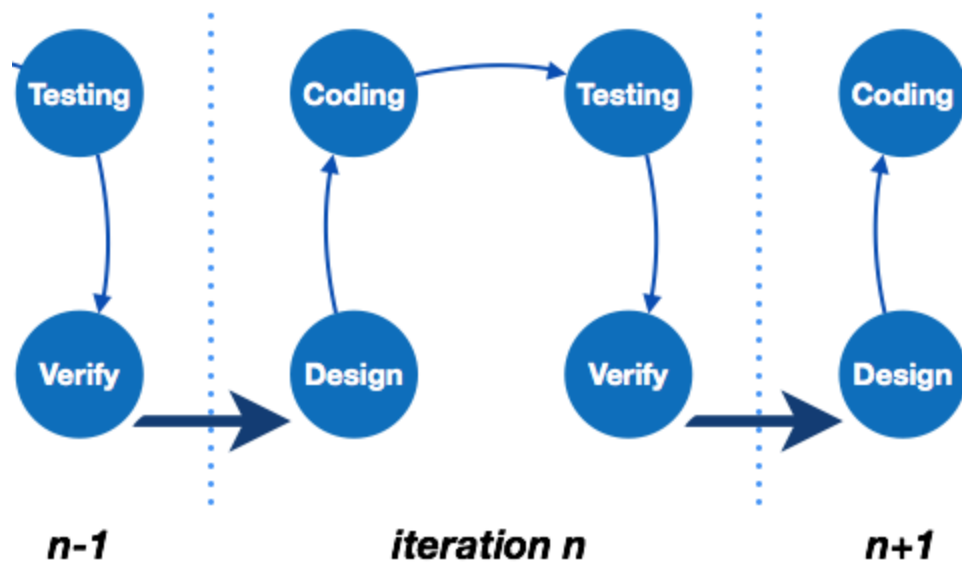
- Errors are easy to be recognized.
- Easier to test and debug
- More flexible.
- Simple to manage risk because it handled during its iteration.
- The Client gets important functionality early.

Disadvantage of Incremental Model

- Need for good planning
- Total Cost is high.
- Well defined module interfaces are needed.

Iterative Model

This model leads the software development process in iterations. It projects the process of development in **cyclic manner repeating every step after every** cycle of SDLC process.



The software is first developed on very small scale and all the steps are followed which are taken into consideration. Then, on every next iteration, more features and modules are designed, coded, tested and added to the software. Every cycle produces a software, which is complete in itself and has more features and capabilities than that of the previous one.

After each iteration, the management team can do work on risk management and prepare for the next iteration. Because a cycle includes small portion of whole software process, it is easier to manage the development process but it consumes more resources.